Building Agentic RAG Systems with Elasticsearch

Hello I am Dishant!

- Work in the field of AI/ML, focusing on Generative AI and Conversational AI
- Enjoy building chatbots and AI tools that solve everyday problems
- Comfortable with cloud and DevOps tools (Docker, Kubernetes, AWS)
- Often share knowledge through talks and workshops with students & communities
- Curious learner who loves simplifying tech concepts for everyone

Table of contents

01 Why ElasticSearch?

02 What is RAG?

03 Hybrid RAG, RRF

04 Kibana

05 Agentic AI

06 Agent Builder

07 Demo

08 QnA

01

Why Elasticsearch?

"You don't need 5 tools to build an AI app.
You need one."

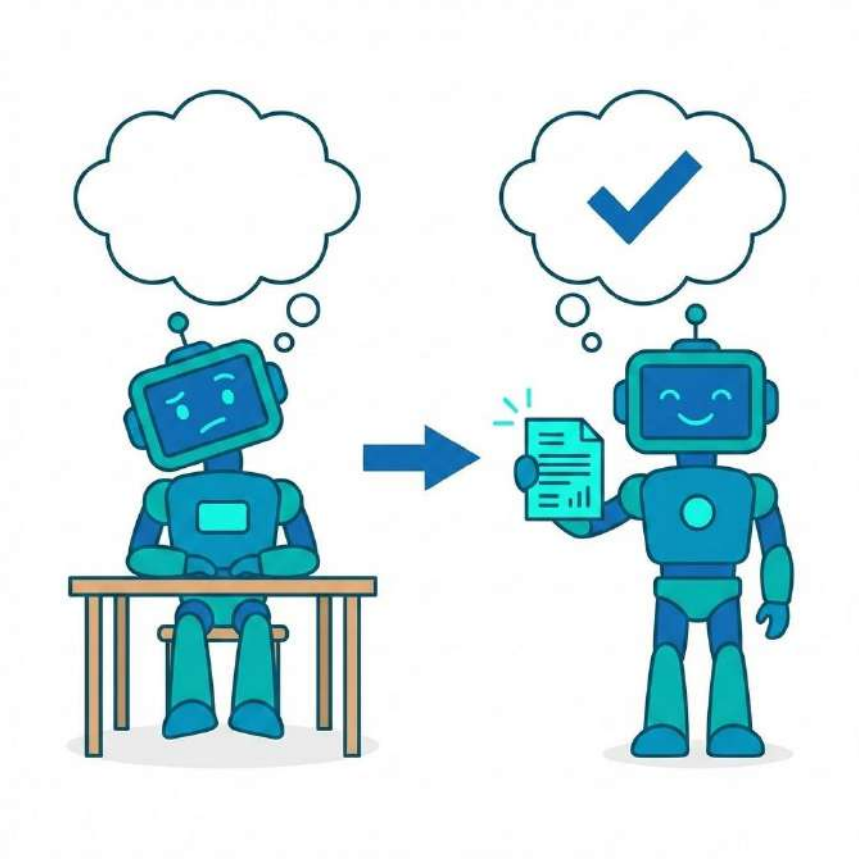
Why Elasticsearch

- Pinecone: cloud-only, expensive at scale, no hybrid search
- ChromaDB: dev-only, not production ready, no observability
- Milvus: complex ops, separate infra, no native LLM tooling
- All of them: you still need 3–4 more tools to build a real app
- With Elastic: one Docker compose file and you're production-ready

02

What is RAG?

What is RAG?

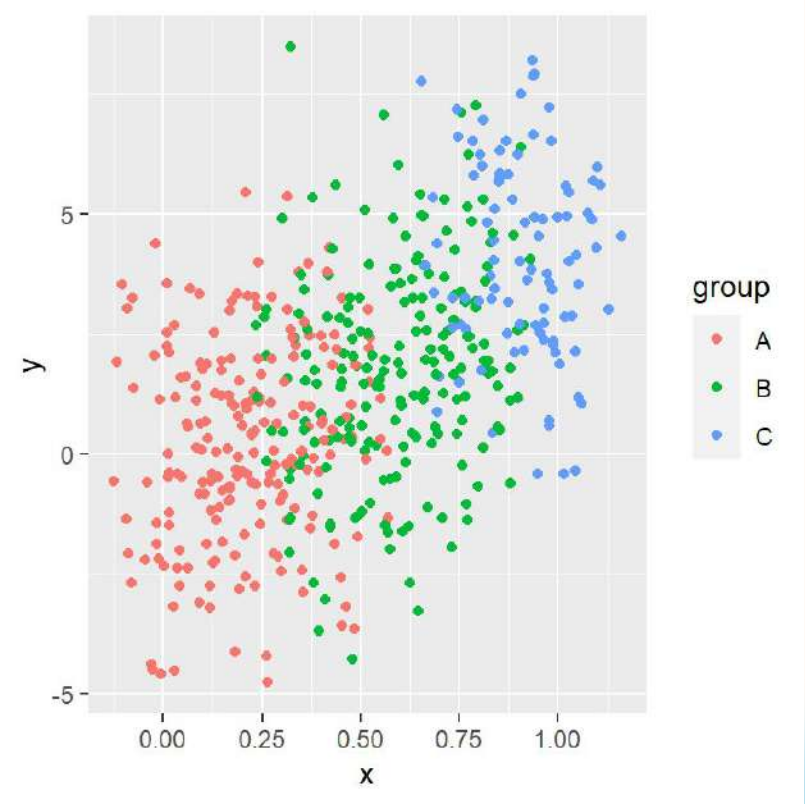


- LLMs are frozen in time — they don't know your data
- RAG = give the LLM a "cheat sheet" before it answers
- The cheat sheet = your documents, retrieved semantically
- Without RAG: LLM guesses → hallucination
- With RAG: LLM reads → grounded answer
- "It's not magic. It's search + generation combined."

What is RAG?

- RAG = Retrieval-Augmented Generation
- LLMs hallucinate without grounding → RAG fixes this
- Pipeline: Documents → Embed → Index → Retrieve → Generate
- Your knowledge base becomes the LLM's context window
- Elasticsearch replaces ChromaDB / Milvus as the vector store

What is Embedding

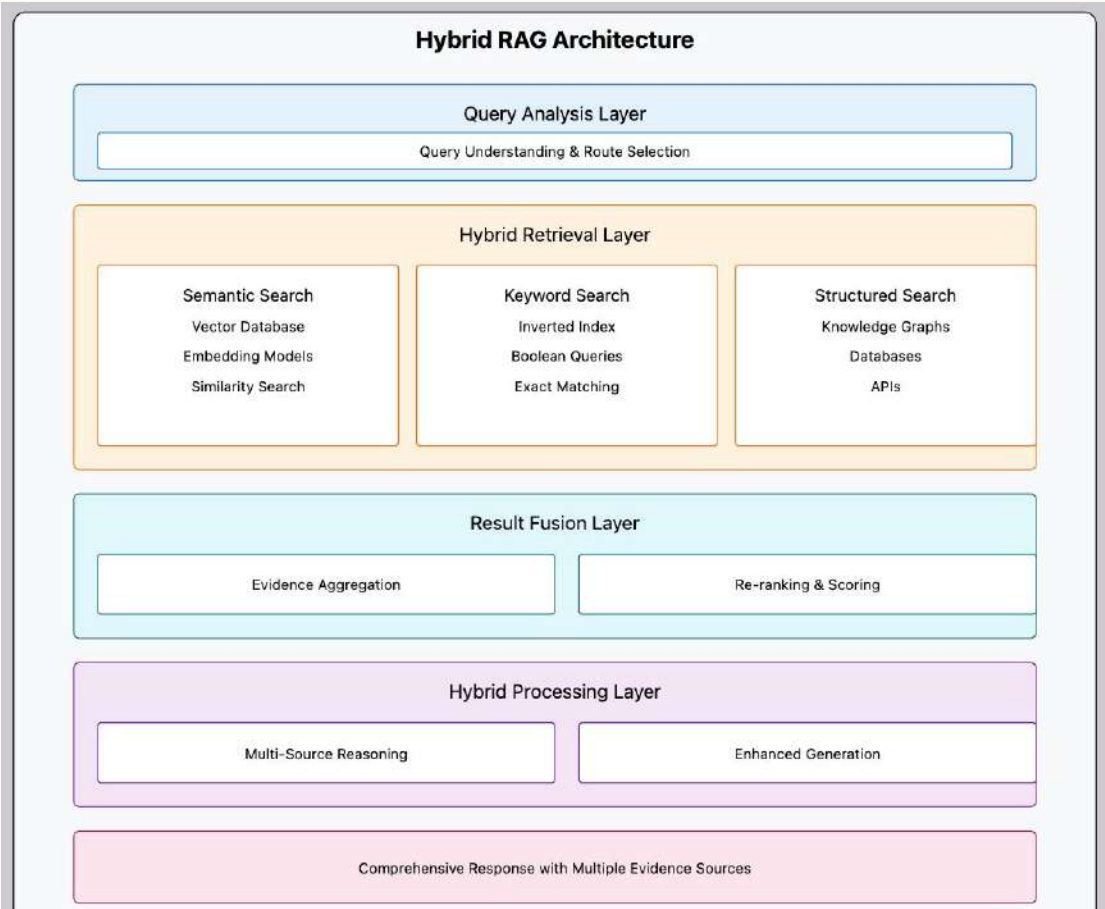


- Words/sentences converted to numbers (vectors)
- Similar meaning = similar numbers = close in space
- "King" and "Queen" are closer than "King" and "Car"
- 384 or 768 or 1536 numbers per sentence
- Elasticsearch stores and searches these numbers at scale
- The embedding model is the translator between language and math

03

Hybrid RAG

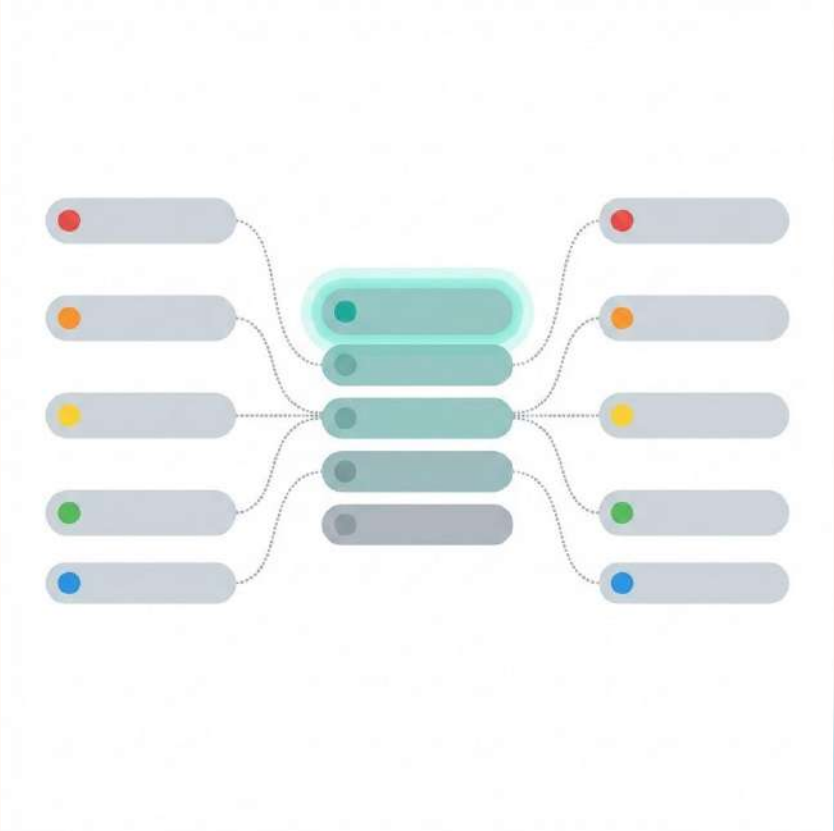
How does it work?



Hybrid search

- Query: "What is the RRF formula in Elasticsearch 8.9?"
- Pure vector search: finds semantically similar docs about ranking — misses the exact version
- Pure keyword search: finds "8.9" and "RRF" — misses the conceptual context
- Hybrid: finds the exact doc about RRF in 8.9 with full context
- This is why production RAG needs hybrid, not just KNN

RRF: How the Merging Works



- BM25 gives ranked list A (by keyword relevance)
- kNN gives ranked list B (by vector similarity)
- RRF formula: score = Σ 1 / (60 + rank)
- A doc ranked #1 in both lists scores highest
- No score normalization needed — ranks are universal
- Result: the best of both search paradigms in one list

AG Quality: What Makes It Good or Bad

Factor	Bad RAG	Good RAG
Chunk size	Too large (loses precision)	256–512 tokens
Search type	Keyword only	Hybrid (BM25 + kNN)
Re-ranking	None	RRF or cross-encoder
Embedding model	Generic	Domain-specific
Context window	Dump everything	Top 3–5 relevant chunks
Observability	None	Kibana traces every query

04

Kibana

Kibana

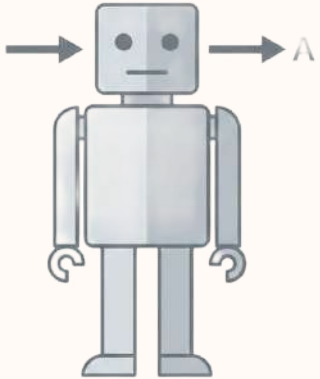
- Kibana acts as the analytics and monitoring interface for the Elastic stack.
- Visualize indexed data in real time
- Monitor search queries and response latency
- Create dashboards for logs, metrics, and traces
- Inspect vector search results and relevance
- Debug ingestion pipelines and document flows

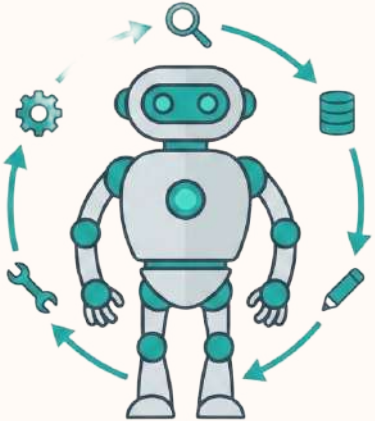
elastic

05

Agentic AI

From RAG to Agentic: What's the Difference?





- RAG: user asks → retrieve → LLM answers → done
- Agentic: user asks → LLM thinks → decides what to retrieve → retrieves → thinks again → answers
- The LLM is no longer a responder. It's a decision maker
- It can loop, use tools, call APIs, query multiple indexes
- Elastic Agent is the orchestration layer that makes this possible

My Agentic AI Stack

- 100% local — no OpenAI, no cloud APIs
- Ollama runs LLaMA / Mistral / Gemma on your laptop
- Elastic runs in Docker
- Total cloud cost: ₹0

How the Agent Decides What To Do

- Agent receives the user query
- Checks: is this a factual lookup or a reasoning task?
- If factual → single ES retrieval → LLM answer
- If complex → multiple retrievals → synthesize → answer
- If uncertain → retrieves more context → re-evaluates
- This loop is what makes it "agentic" vs basic RAG
- "The LLM is the brain. Elasticsearch is the memory."

Why Local LLM + Elastic is Powerful

- Ollama runs Qwen3.5/LLaMA3/Gemma4 locally
- No data leaves your machine — 100% private
- Elastic stores and retrieves your private documents
- Together: a fully private, offline, intelligent RAG agent
- Use cases: internal docs, medical records, legal files, code repos
- Enterprise-grade privacy with zero cloud dependency

What My Agent Can Do

- Answer questions from indexed PDF documents
- Cite the exact source chunk it retrieved
- Handle follow-up questions with context memory
- Run entirely offline on a MacBook / Windows machine
- Observed in real time via Kibana dashboards
- Swap the LLM model in one line (Qwen3.5 → Gemma4)

06

Agent Builder

You Don't Have to Code This

- Everything I built manually → Elastic Agent Builder does visually
- No Python, no embeddings code, no Docker compose knowledge needed
- Point-and-click interface inside Kibana
- Connect your data source → choose your LLM → deploy
- Elastic handles chunking, embedding, indexing, retrieval automatically
- "What took me 3 days of setup — takes 20 minutes in Agent Builder"

07

Walkthrough

08

Ask! Ask!

Lets Connect!

Thanks!

Credits: All icons and images were generated downloaded from AI and slidesgo